

Framework for a Digital Age Verification System using Cryptographic Signatures and the Ethereum Blockchain

Tristan Mocio, Zack Dankenbring, Bryce Bullard

May 2, 2026

It is recommended that readers complete the **Logic** portion of this paper before running the code. If you would like to skip to the code, go to the section: **Instructions For Running Code**.

Background

Digital age verification has become a major topic in recent years. Many legislators are attempting to make it a requirement for accessing apps, sites and even operating systems. This leaves developers scrambling to find ways to become compliant. Often at the detriment of privacy and security. Because of this, I developed a system that could verify a users age effectively, securely and efficiently without sacrificing user privacy.

Existing Age Verification Systems

I began by doing a brief investigation into what systems currently exist for digital age verification. There are dozens of systems used across the internet, but I compiled a few that I felt were worth discussing:

- **Self-Declaration:** Perhaps the most common method, sites/apps just ask the user to self-report their age. This obviously has a major flaw in that it is entirely ineffective at accurately determining age. It puts full and complete trust on users to be honest when reporting.
- **ID-Submission:** Another common method, users submit an image of their ID and sites/apps pull their information from the ID photo. This also has major flaws. It allows the site access to a ton of unnecessary information about the user, and it is extremely easy for users to borrow someone elses ID and submit it. Or users can pull a photo off of the internet and pass it off as themselves. Not only that, but verification is site exclusive, meaning that users have to verify independently on every site that requires age verification. This means that users have to submit their ID to every single site or app they want to use.
- **Identity Tokens (OpenID Connect):** This system was discovered after the completion of this project. It shares some similarities to the system I devised, but it has some very notable differences (important to note that these systems were created independently). First, a user will go to a trusted issuer (like google or apple) and register, creating an account that contains a slew of personal information. Then when they want to verify on a website, they select an option like 'sign-in with google'. Then they will enter their issuer account information (like their google account details) and the issuer will send the app/site their information as a token. The site will use the token to verify their age and permit or deny access. There are huge issues with this system not present in my own. Firstly, the system is centralized. You must go through specific issuers both to create an identity and verify. This keeps the issuer heavily involved in all parts of the system allowing them to track users across sites. It also often gives sites nearly the full account in the token, meaning they get access to a ton

of user information. Accounts are also stored centrally by issuers creating risks of data leakage. Also, the system requires verification info to be transmitted between parties, creating risks of interception by malicious individuals. Lastly, how do issuers verify age? When users create an account through a provider like apple or google, the age verification process done by those providers is not rigorous. I believe that our system has clear advantages over Identity Token systems.

Overview

This age-verification system was devised as an alternative to the first two systems listed above (self-declaration and ID-submission). I felt like we could come up with a system that was more accurate at gating access than Self-Declaration and simultaneously more private and secure than ID-Submission. As previously mentioned, ID-Submission systems require users to submit their ID to every site that requires age verification that they want to access. This means verification is fully decentralized, to users detriment. If a user has to submit their government ID to dozens of sites and apps, this creates dozens of failure points where users data can be shared and stored by hackers or sites themselves. But why would users have to verify users age independently across sites? Why not have users verify their age once, and then just prove to each site that the user has already verified their age? That would provide an upper bound on the amount of trust required by users. Instead of creating dozens of failure points, we can bound it to 1 failure point, that being the organization that initially verifies their age. Then, we can impose strict standards for the initial verifier regarding both what they do with user data, and the accuracy of their verification process. Thus users can trust that their personal data will be handled securely, and sites/applications can trust that the decision to gate access to users is made accurately. Using Cryptography and the Ethereum Blockchain we can build this system in a manner that is decentralized, scalable, and secure.

File Structure

Included with this report are 3 files.

- **ageVerify.py** This file handles all of the off chain operations for all parties. In practice the operations would be broke up and each party would do its own sections independently, but for demonstration purposes it is self contained here. It writes its final output as a JSON file.
- **ageVerification.sol** This file handles all of the on-chain operations. In practice it would be deployed on the Ethereum blockchain and all on-chain computations of the system would be done through this contract.
- **verifyAgeDeploy.py** This file is used in the demonstration of the system. It allows a user to deploy the contract on a local QBFT-based blockchain network using Hyperledger Besu. It also parses the JSON output of **ageVerify.py** and uses it to interact with the contract. I used this file to test the smart contract on my own machine.

Both `ageVerify.py` and `ageVerification.sol` include a lot of comments to explain each function and how the contract itself works. These comments, along with this README should fully explain the system.

Logic

The system relies on interactions between 4 parties, listed below:

- **The Issuer:** An individual or organization that can verify individuals age. This is the ‘root’ of age verification.
- **The User:** Anyone that wants to have their age verified.
- **The Verifier:** Anyone that wants to verify the user’s age (sites and applications).
- **The Owner:** The organization, individual or community that controls the contract and the standards placed on issuers. This can be anyone, a government, company, community wide vote, etc.

Before the system can function, `ageVerification.sol` must be deployed on the Ethereum blockchain by the owner. Then, issuers will generate an ethereum address and private key pair. This will be stored by the issuer, their private key must remain secure. Then the issuer must prove that they meet some security standards. I discuss these standards in the **Certification System and Registry** section later. Once the owner has determined that an issuer meets these standards, the issuer can be considered ‘trusted’. Once an issuer is considered trusted, they will have their Ethereum address added to a registry of trusted issuers using the `addAddress` function in the `ageVerification.sol` smart contract. Currently this function can only be called by the deployer of the smart contract but it can be modified to be done by whoever is considered the owner using the `transferOwnership` function.

Next, a user can create their own Ethereum address and private key pair. This must also be stored securely by the user. Any user who wants to take part in the system must then contact a trusted issuer. But how does a user know that the issuer is actually trusted? We need a system to allow parties to prove who they are, otherwise anyone can impersonate issuers and claim to be trusted, when they are not. We could have issuers provide their private keys to users. Then users can verify the pair and assume that since the individual possesses a trusted address and the corresponding private key, they are a trusted issuer (more on this assumption in the **Future Enhancements** section). But, obviously that private key would no longer be ‘private’ and the system would break. Thus we need a way for people to prove that they possess the private key that corresponds with a given address, without providing the key itself. It turns out this is actually a simple problem to solve. We have users generate a random integer (called a nonce), then they ‘challenge’ the issuer to use their private key to sign the nonce and send the signature back to the user. Then the user can then verify the signature, and determine whether or not the issuer is trusted. This can be done entirely off-chain. I also included a function in `ageVerification.sol` called `issuerVerify` that users can use to verify the signature. This challenge system is an effective way for individuals to ‘prove’ that they possess a private key without compromising it. Once the

challenge is verified, the user can then confidently submit their personal information to the issuer. Because they know that the issuer meets the security requirements to be considered ‘trusted’. This information can be whatever is necessary to allow the issuer to accurately determine their age along with the user’s Ethereum address. Because the issuer is certified, the privacy concern with sharing this information is inversely correlated with the strength of the ‘trusted’ certification. The stricter the certification process, the more comfortable users will be with sharing their personal information. This certification process is a direct bound on the amount of trust required for the entire system.

Once the user has sent their personal information to the issuer, the issuer can then accurately determine their age and determine the result of the boolean `is18plus`.

$$is18plus = \begin{cases} \text{True} & \text{if } age \geq 18 \\ \text{False} & \text{if } age < 18 \end{cases}$$

We use a boolean ‘is18plus’ instead of an alternative, like encoding their birth year, because we want to limit the amount of information given to sites. Whether or not the user is 18 plus, or not, is the only information necessary for a site or app to grant or deny access. This does mean that the system is currently only capable of determining their status in relation to 18. It cannot prove whether or not a user is any other age, like 13 or 16 or 21. Issuers could add more booleans, or encode the year of birth to verify conditions other than `is18plus`. I talk more about this in the **Future Enhancements** section. This also means that users would have to resubmit their personal information and redo the process if they turn 18

Next, the issuer must also generate an expiration date. This date can be determined by the issuers, there are no requirements placed on its value as long as it is represented as an integer in unix time. I will go into more detail about the inclusion of expiration dates later on. Once the issuer has determined the value of the boolean, they can create an attestation. The attestation is a signed hash containing everything a site or app would need to verify the users age. It contains:

- The users Ethereum address
- The boolean `is18plus`
- The generated expiration date (as an integer)

Below is sample code for how an attestation is hashes, then in python.

```
# The hash requires an address, the boolean and the expiration
attestation_hash = Web3.solidity_keccak(
    ['address', 'bool', 'uint256'],
    [user_address, is18plus, attestation_expiration]
)
# The issuer then uses their private key to sign the attestation
issuer_signed = Account.sign_message(
    encode_defunct(attestation_hash), issuer_private_key
)
issuer_signature = issuer_signed.signature
```

We use keccak-256 to hash the attestation as it is the standard algorithm for Solidity.

Here is an example of a sample attestation:

--- ATTESTATION ---

```
User Address:      0x413f997D0D404643628bfd45b3Ee3F5A8cC44944
Is 18+:            True
Expiration (unix): 1808950747
Issuer Signature:  41cf1daab7e19e26421ef....697794ac3cdf680447392b168b81679f1c
```

Once the attestation is created, it can then be sent to the user along with the issuers Ethereum address. The user will then securely store the entire attestation, and the issuers address.

Now the user possesses a statement containing their age status that was signed by someone who we know verifiably is able to accurately and safely determine users age status. Since we know that valid attestations can only be made by issuers that are considered ‘trusted’, users can feel more comfortable sharing their personal

Logic: Part 2

For the purposes of age verification, the attestation can act as a digital ID card. It can identify a user and their age status through the Ethereum address and boolean status.

If a user wants access to a site or application that requires age verification, they first will prompt the site. Normally, the user would then send their attestation to the site, but how do we know that the address inside the attestation corresponds with the user that is currently trying to verify? To determine this, we use a challenge much like we did previously to verify that the issuer is trusted. When prompted for verification, the site will automatically generate a nonce, send it to the user, and ask them to sign it.

Below is sample code for the signature process:

```
nonce_hash = Web3.solidity_keccak(['uint256'], [nonce])
user_signed = Account.sign_message(encode_defunct(nonce_hash), user_private_key)
user_challenge_signature = user_signed.signature
```

Once the nonce has been signed, the user can then package all of the data and send it to the verifier. Below is a sample of the data that is sent to the verifier:

--- DATA SENT TO VERIFIER TO VERIFY ---

```
User Address:      0x413f997D0D404643628bfd45b3Ee3F5A8cC44944
Issuer Address:    0xf876c74cA53673dC7E5E91001Ad9B5c2731f94b5
Is 18+:            True
Expiration (unix): 1808950747
Issuer Signature:  41cf1daab7e19e26421ef....447392b168b81679f1c
User Challenge Signature: 38ac3fe10748b1e2d981e....4beaf69168faae51a3c
```

The verifier can then take this data, and add the nonce they originally sent. In the attached code it was done using JSON. Here is an example of a JSON package:

```
{
  "issuer_address": "0x9e64EE4b7Bd4cEF9815b6AeB02829cab2697878B",
  "user_address": "0x4EE84EbcE538014aF09B60d1DbA299912F8B4866",
  "is18plus": true,
  "nonce": 4868984341717489079073400607107496081203356648573303898483662525057774649134,
  "attestation_expiration": 1809144445,
  "issuer_signature": "d7cd1de7fcb2cde672d2177fdcccb31b812ae4ffffeca240e8f177f13d2f1ffa7",
  "user_challenge_signature": "571fcfebb1044e8ac046ef95ef5d6eb6dabd98cbc597d7b894babca90"
}
```

Logic: Part 3

Now that the data is received by the verifier and packaged, we can finally verify it. To do this, the verifier will call the `verifyAge` function in `AgeVerification.sol`

```
function verifyAge(
    address issuer,
    address user,
    bool is18plus,
    uint256 nonce,
    uint256 attestation_expiration,
    bytes memory issuer_signature,
    bytes memory user_signature
) public returns (bool) {
```

If you look at the JSON package previously, and the above solidity function, you'll notice that they take the exact same arguments. This allows the verifier to directly submit the JSON package and have it parsed and read by the contract.

The `verifyAge` function checks several conditions in order to determine if users should be granted access or not. It:

- Checks that the issuer address listed is considered trusted.
- Checks that the issuer-user address pair is not considered revoked.
- Checks the signature expiration against the current time to ensure that the attestation isn't expired.
- Hashes the original attestation and verifies it was correctly signed by the issuer.
- Hashes the nonce and verifies that the user signed it correctly.

If all of these conditions are met, the function passes and returns the `is18plus` boolean. It also emits an event with the information given to the function:

```
event AgeVerified(address indexed user, address indexed issuer, bool is18plus);
```

Sites can use this event to determine whether or not the user can be granted access to the restricted content. If the function fails, the user can be denied.

Instructions For Running Code

Upon completion of the logic portions, readers should:

- Run `ageVerify.py` this file has comments explaining each step of the process. It writes its final output as a JSON file.
- Start a local blockchain network, this can be done using HyperLedger Besu.
- Run `verifyAgeDeploy.py` this file allows the smart contract `ageVerification.sol` to be compiled and deployed on the local blockchain network. It also parses the JSON file output by the previous file and uses it to interact with the functions within the contract.

Security Features and Limitations

Now I will discuss the security enhancements and known flaws that are included in the system, most of them residing in the solidity contract.

It is important to keep in mind that the system is not designed to be perfectly foolproof, just better than the status quo. There were several deliberate security sacrifices made to allow the system to be streamlined enough to be usable and scalable.

The most important security feature, is the most intrinsic.

Cryptographic Signatures and Hashing (keccak-256 and ECDSA)

The core of this system relies on using ECDSA to cryptographically sign pieces of data. Once to challenge the issuer, once to create the attestation and once to challenge the user, for a total of 3. These signatures are extremely useful because they allow us to mathematically prove that a piece of data was created, or at least verified, by a specific individual. The signature allows us to prove beyond a reasonable doubt that a signature was created by the private key associated with the signature address. Thus if you receive a signed attestation, you can mathematically prove that it was signed by the issuers private key. If the issuer is considered trusted, then you know that someone whom you trust to accurately verify age, has verified an individuals age. We use challenges to let individuals prove their identity by signing something. If a user signs a number, and we verify the number with their address, we can prove that whoever just prompted us for verification does in fact possess the private key associated with the given address. We also use challenges for issuers. How does a user know that the issuer they are about to give their information to is actually trusted? They can do the same thing. Send them a number, have the issuer cryptographically sign it, and then verify it to prove that the issuer has the private key corresponding with a trusted issuer address. An alternative to this would be just having a party give their private key. But this is both unnecessary, and creates risk of interception or theft, and so we avoid it by having them sign something instead. This way we prove that they possess the private key, without receiving the private key. The elegance of this system is that aside from the initial

verification step with the issuer, no private information is transmitted. Private keys are never transmitted between any parties. The only data that is transmitted between parties is data that can exist publicly.

Certification System and Registry

Unfortunately, age verification cannot be done purely trustless. Nearly all systems require a root individual to verify age initially. This is often done through a system called KYC (Know Your Customer) which is a rigorous standard for overall identity verification often involving users submitting a massive amount of personal information, like photos of government documents. In order to use a system like this, we use a certification that organizations that want to issue attestations can obtain. This certification will grant them the ‘trusted’ status. The ‘trusted’ status must be bipartite, it must enforce correctness in age verification and attestation creation, and also security with regard to user personal information and data. Once an attestation is sent to the user, the issuer can fully delete all records of the individual, this should be a requirement to obtain ‘trusted’ status. But how do we determine what organizations have obtained ‘trusted’ status in a decentralized system? We do this through solidity functions built in to `ageVerification.sol`.

The functions `addAddress` allows the contract owner to add an Ethereum address to a registry of issuers that are considered ‘trusted’.

```
function addAddress(address _address) public ownerOnly {
    require(!isIssuer[_address], "Already added");
    isIssuer[_address] = true;
    emit IssuerAdded(_address);
}
```

We also incorporate the function `revokeAddress` which allows the contract owner to remove an Ethereum address from the registry should an issuer lose ‘trusted’ status.

```
function revokeAddress(address _address) public ownerOnly {
    require(isIssuer[_address], "Not in registry");
    isIssuer[_address] = false;
    emit IssuerRevoked(_address);
}
```

We also include a function `isTrusted` which allows anyone to explicitly check if an address is in the registry.

```
function isTrusted(address _address) public view returns (bool) {
    return isIssuer[_address];
}
```

Using solidity to hold the registry information is extremely useful because it ensures that anyone can check which addresses are trusted and not have to worry about receiving inaccurate or outdated information.

Nonce Security

In each of the challenges, we require the transmission of a nonce (just a randomly generated integer). Each of the challenge nonces are generated independently and off chain. There is only one suggested requirement of the nonce, and that is that it is not reused. The goal of the challenge system is to force the individual being challenged to sign something at runtime. If nonces are reused the challenged party could reuse an old signature and fake sign something. This would fail to effectively determine if they possess the private key. We could enforce this by storing nonces in the contract itself and rejecting function calls that contain nonces that have been used previously. There are several reasons I chose not to do this. The first being that it can induce collisions. Where an honestly generated nonce happens to match with one that was used previously and thus a valid challenge would fail. Secondly, it is extremely expensive to write to memory on the blockchain. Because of this, I chose to put the responsibility of generating nonces honestly on the individual that is sending the challenge. Another issue is that the nonce is not generated on the blockchain, as I mentioned it is entirely on the challenger to generate the nonce, and input it into the solidity function call correctly. A malicious challenger could enter a different nonce than they sent the challenged party. They could let the challenged party come up with their own nonce. This would cause the challenge to become ineffective entirely. However, similar to the reasons I chose not to store past nonces, putting the requirement on the individual issuing the challenge to generate the nonce is a deliberate choice to make the system efficient. It is possible to have the nonce generated by a smart contract, and the inputted attestation checked against that nonce, but this was deliberately left out. In short, the system assumes the verifier is honest, which I believe is a fair assumption because a malicious verifier could falsify any part of the process already.

Contract Ownership

This was touched on previously, there are several functions that can only be called by the owner of the contract (`addAddress` and `revokeAddress`). This creates a classic oracle problem, we do not want a central authority to control who is considered trusted. This is left intentionally open. The certification process to obtain ‘trusted’ status is up for debate, and whoever controls that process can also control the contract. There is a function in `ageVerification.sol` called `transferOwnership` that allows the deployer of the contract to transfer ownership to a different address.

```
function transferOwnership(address newOwner) public ownerOnly {
    require(newOwner != address(0), "Invalid address");
    owner = newOwner;
}
```

When the ‘trusted’ status is determined by either legislators, a community, etc, the owner can then be decided.

Fraudulent Verification

Initially, as long as an attestation was initially made correctly, the issuer address is considered ‘trusted’ and the user possess the associated private key, attestations would verify indefinitely. This is a major flaw. If a user has their private key stolen, or deliberately shares it, their attestations could pass. I feared a situation arising where there are massive online libraries of valid attestations for anyone to grab and use. To remedy this, we have two separate systems. First, I include an expiration date inside of the attestation in unix time. This is an arbitrary value decided by the issuer. It can also have explicit guidelines that issuers adhere to to obtain trusted status. This provides an upper bound on the damage caused by stolen or shared keys and attestations, as they will only be usable for a set amount of time. The `verifyAge` function in `ageVerification.sol` checks the expiration date against the current time as part of its verification process. The second system to prevent fraudulent verification is built in to the contract natively. If an issuer has created widespread fraudulent attestations, they can have their address pulled from the list of trusted addresses, generate a new one, and have it added to the list. This would cause every attestation made under that issuer address to fail verification. This also means that any valid attestation made under that address would also fail verification. Thus it is intended to be used as a last resort. The third system is a direct revocation of attestations. This is done through the `revokeAttestation` function in solidity.

```
function revokeAttestation(address _user) public issuerOnly {
    require(!revokedPairs[msg.sender][_user], "Already revoked");
    revokedPairs[msg.sender][_user] = true;
    emit AttestationRevoked(msg.sender, _user);
}
```

This function allows any issuer that is considered trusted to explicitly revoke an attestation that they made if it is found to be shared, or fraudulent in some way. But issuers do not store the attestations that they made, so how can they explicitly revoke one? This function works by taking a user, issuer address pair. If a trusted issuer calls this function, they must send the address they want to deny as an argument. The function will then take the address that called the function, and instead of denying that address from verifying altogether, it denies the issuer, user address pair from verifying. This means that an issuer cannot deny a user’s verification unless they are attempting to use an attestation that was made by the same issuer (in simpler terms, issuers can only revoke attestations that they made). This prevents rouge issuers from denying other issuers attestations.

Blockchain Related Considerations

A major concern with using solidity smart contracts is gas costs. Deploying and using a smart contract incurs fees. Thus throughout development I made considerable sacrifices to minimize these costs. Things like storing attestations on chain would allow for more security and control, but it would also incur heavy costs making the system infeasible. The contract itself is mostly made up of function calls that perform computations and do not write to

memory. While this does incur costs, it is relatively inexpensive, although there is always room for optimization. There are two functions that do write to memory. These are:

- **addAddress** This function adds a boolean mapping to an address. Essentially it takes an Ethereum address and maps it to **True**. This mapping is how we determine if an issuer is trusted or not. If the contract owner wants to declare that an issuer is now able to issue attestations, they can call the function and provide the address, thus mapping it to true. This requires storing the mapping as persistent storage. Which does incur some gas fees, but because it is purely a mapping, the cost is relatively low. From reading the Ethereum documentation, I believe that deploying the contract on the Ethereum mainnet would incur roughly ~22,000 to ~25,000 gas per function call. Converting to USD this can cost anywhere from a few cents to a few dollars. While this is definitely notable, I believe it is feasible for the system to operate with this cost. (There is also a **revokeAddress** function that does the reverse and removes the True mapping).
- **revokeAttestation** This is the most expensive function in **ageVerification.sol**. It allows an issuer to explicitly revoke an attestation that they made. It does this by allowing an issuer to call the function, and provide an address. The function then stores a boolean mapping on both the address included in the function, and the address of the issuer that called the function. Then, whenever a user tries to verify their age, the **verifyAge** function will check the user address, and the issuer address to see if the *pair* has been revoked. This is so that issuers can only revoke attestations that they made. This requires a bit more storage than **addAddress** as the contract must map the user issuer address pair to a boolean. As mentioned previously, this function is meant as a last result and not meant to be used regularly.

Another consideration that comes with the Ethereum blockchain is that calling functions takes time. Users of a smart contract must wait for the result to be computed and included in a block. The block time for Ethereum is just 12 seconds, but it can take longer depending on traffic. That being said, I believe the delay is manageable, and an acceptable drawback of blockchain based systems. (as an aside, some ID-submission systems require manual human verification of ID data where a human must look at the photo submitted and then determine if it should be accepted or not. Having some degree of latency or delay in age verification systems is not unheard of.)

A major benefit of using the blockchain for this system is that it allows the system to adapt and evolve over time. Should organizations want to alter the system, they can deploy a new smart contract and route verification through it.

Future Enhancements

- It is possible to swap the **is18plus** boolean for anything. In theory we could use this to allow users to verify any condition, not just whether or not they are older than or younger than 18. Any other age could be encoded, but not only that, conditions can be encoded. You could use this system to prove things like certifications. If a user wants

to purchase something that requires training. They could get an attestation proving that they have received that training, and then present it to the site.

- I spoke previously about the decision to include the `is18plus` boolean and its advantages and drawbacks. But there is an alternative that could mitigate a lot of its drawbacks. We could replace the boolean with a Zero Knowledge Proof. Allowing verifiers to prove that a user meets a certain condition without revealing *any* information about that user. I believe there is a clear application here for Zero Knowledge Proofs, although more work would need to be done in this regard.
- This system relies on a major assumption. We assume that whoever possesses the private key that corresponds with a given address is actually the individual the address corresponds with. If users share private keys, they can effectively impersonate each other. This is definitely a concern, and a hard one to address. Traditionally the Ethereum blockchain has a built in incentive not to share keys, if a user shares their private key they could have their wallet emptied. Since this system does not involve users holding or transacting with ETH, that incentive does not exist. However, the current age verification systems all suffer from the same flaw. Users can always share google account info, or physical ID cards. There is potential to add an incentive structure to drive people to keep their private keys hidden, like having users pay a small deposit that can be redeemed when the attestation expires. But I left this as a known limitation of the system.
- A through-line in this system is that it forces all parties to interact with complex systems they may have no familiarity or knowledge on. Cryptographic and Blockchain based technologies can be difficult for the average user. This system would benefit from those processes being abstracted away through something like a dApp (decentralized application) which could help make the system easier for users, issuers and verifiers.
- I have spoken about the contract owner, and oracle problem briefly. If the entire system hinges on issuers being ‘trusted’ who determines which issuers do and do not fit in this category? I did not specify who would actually control the contract deliberately. As age verification is a complex problem. The solution needs to be accurate enough for legislators, secure enough for users, and efficient enough for sites and applications all at the same time. More development would need to be done in this field through working with legislators to determine who can control the contract and the ‘trusted’ status. This could be done through the creation of a DAO (decentralized autonomous organization). Regardless it is a question that is still left unanswered.
- The system exists on a spectrum between security and usability. Because of this, organizations can alter it in one way or another and create different versions of the system that meet different goals. If there is a system that must enforce stricter accuracy regarding age, issuers can create a more advanced system that uses stricter verification processes. Or vice-versa, should an issuer want a system that is faster and easier to use, a lighter version can exist that is less accurate.